

Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

QUINTO CICLO "A"

ASIGNATURA: Sistemas Digitales

ELABORADO POR:

Anderson Mateo Coello Jaramillo

Docente encargado: Ing. Luis Sinche

LOJA – ECUADOR

Mis apuntes de clase

Sistemas Digitales · Unidad 1 · Semanas 1–4

> **En general:** Microcontroladores para sistemas embebidos → primero cómo está hecho el chip; luego cómo escribir bits, manejar GPIO, medir tiempo y atender eventos.

↳ *Formato: apuntes pasados en limpio; no es informe de laboratorio ni transcripción literal de diapositivas.*

SEMANA 1 / DEL COMPUTADOR AL MICROCONTROLADOR + BITS

↳ *Clase → lo esencial, con abreviaturas y ejemplos. Algunas ideas quedan como en el cuaderno...*

Sesión 1 · Lo básico del hardware

→ Sistema embebido → computador dentro de otro aparato. Función concreta, recursos limitados y a veces tiempo de respuesta importante. Ej.: lavadora, alarma, router, sensor...

→ Microprocesador ≈ CPU enfocada en procesar; en un PC se acompaña de memorias y dispositivos. Microcontrolador (MCU) → CPU + memoria + periféricos integrados en un chip.

→ Ejemplos vistos: ATmega328P (Arduino Uno) y ESP32-WROOM-32. AVR 8 bits; ESP32 clásico 32 bits, 2 núcleos.

→ Transistores → sirven para conmutar o amplificar. En un BJT: base, emisor, colector. Con ellos se construyen puertas y circuitos digitales...

→ Lógica combinacional → salida depende de entradas actuales (AND, OR, NOT, XOR). Secuencial → hay memoria del estado; flip-flops, registros, contadores.

→ ALU = unidad aritmético-lógica → sumas, restas, AND, OR, XOR. Unidad de control → coordina. Registros → almacenamiento rápido dentro de CPU.

→ PC (Program Counter) → dirección de próxima instrucción. CPU: buscar → decodificar → ejecutar... y repetir.

> **Reloj:** 16 MHz = 16 millones de pulsos por segundo. Tiempo de un ciclo de reloj = 1/16 000 000 s = 62,5 ns. Frecuencia no es directamente "instrucciones por segundo".

Buses: tres cosas diferentes

Bus	Idea que llevo
Direcciones	¿En qué posición? → 0x25
Datos	¿Qué valor? → 0010 0000
Control	¿Qué acción? → escribir / leer

↳ *Ejemplo para recordar: escribir 0010 0000 en 0x25 → direcciones 0x25, datos 0010 0000, control "escribir".*

Memorias, Harvard y von Neumann

→ Von Neumann → memoria/camino compartido para instrucciones y datos; hay competencia por el acceso.

→ Harvard → caminos separados para programa y datos; se pueden buscar instrucciones mientras se usan datos. ATmega328P → Harvard.

Memoria	Qué guardo / qué pasa sin luz
Flash	Programa → no volátil; se reprograma
SRAM	Variables durante ejecución → volátil, se pierde
EEPROM	Pocos datos cambiantes que deben permanecer → no volátil

→ ATmega328P: Flash 32 KB, SRAM 2 KB, EEPROM 1 KB. En módulo ESP32-WROOM-32 de clase: Flash externa habitual 4 MB y SRAM interna aprox. 520 KB; EEPROM física no como AVR.

→ Chip ≠ módulo ≠ placa. Chip ESP32; módulo WROOM añade Flash y antena; DevKit añade USB, regulador, botones, pines.

↳ *1 byte = 8 bits; 1 KiB = 1024 bytes. Cuidado: Mb (bits) ≠ MB (bytes).*

Sesión 2 · Operaciones bit a bit

→ Bit = 0 o 1. Byte = 8 bits. Binario 0b... ; hexadecimal 0x... ; decimal sin prefijo.

```
128  64  32  16   8   4   2   1  ← pesos del byte
  1   0   1   0   1   0   1   0  = 170 = 0xAA
```

Operación	Cuándo da 1	Uso
AND &	Solo si ambos son 1	Consultar / limpiar bits
OR |	Si al menos uno es 1	Encender bits
XOR ^	Cuando son distintos	Alternar bits
NOT ~	Invierte cada bit	Máscara invertida

Máscaras (memorizar): ENCENDER: REG |= (1 << n); APAGAR: REG &= ~(1 << n); INVERTIR: REG ^= (1 << n); LEER: if (REG & (1 << n)).

→ 1 << 5 = 0010 0000; el 1 se mueve cinco posiciones. Ojo con contar bits desde 0.

```
REG = 0b10101010;
REG |= (1 << 0);      // 10101011
REG &= ~(1 << 7);     // 00101011 (si sigue del anterior)
REG ^= 0x0F;          // 00100100 (si sigue del anterior)
```

↳ *Si cada ejercicio dice "partiendo del valor inicial", reiniciar REG; si son instrucciones seguidas, usar el resultado anterior.*

Registros AVR: DDR, PORT y PIN

Registro	¿Qué hace?	Bit = 1
DDRB / DDRD	dirección del pin	salida
PORTB / PORTD	escribir / pull-up	alto o pull-up activa
PINB / PIND	leer voltaje del pin	HIGH

```
DDRB  |= (1 << 5);   // PB5 salida
PORTB |= (1 << 5);   // PB5 HIGH
PORTB &= ~(1 << 5);  // PB5 LOW
PORTB ^= (1 << 5);   // PB5 cambia
```

→ PORTB = (1 << 5) sobrescribe todo el registro; PORTB |= (1 << 5) conserva los otros bits.

→ Botón PD2 con pull-up: DDRD2=0 (entrada), PORTD2=1 (activar resistencia). Suelto → PIND2=1; presionado a GND → PIND2=0.

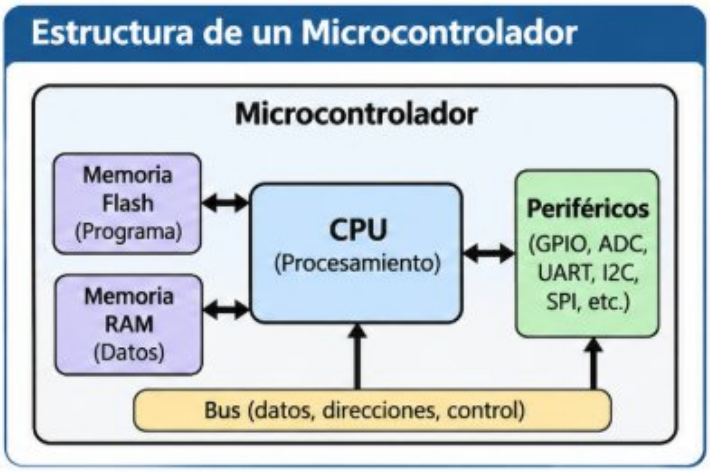
Ensamblador, RISC, CISC y APE 1

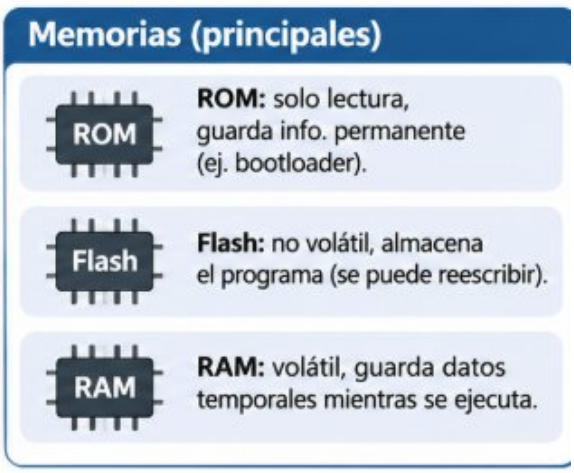
→ Compilador → C/C++ se transforma en instrucciones de máquina. En AVR aparecieron: LDI (carga inmediato), ADD, SBI (poner bit), CBI (borrar bit), RJMP (salto), IN/OUT (E/S), ORI/EOR (lógica).

→ RISC: conjunto reducido/simple de instrucciones; CISC: repertorio con instrucciones más complejas. No confundir RISC=MCU y CISC=PC.

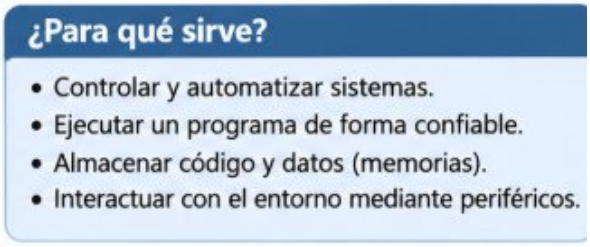
→ Archivo .lss de la práctica: para |= de PORTB se observó IN, ORI, OUT (3 ciclos); para ^= apareció IN, LDI, EOR, OUT (4 ciclos). Otro compilado puede usar SBI según optimización.

↳ *Idea que quedó: las APIs simplifican, el registro da más control... pero menos portabilidad.*









SEMANA 2 / ESP32, TIPOS, MILLIS() Y PULSADORES

↳ *Clase → lo esencial, con abreviaturas y ejemplos. Algunas ideas quedan como en el cuaderno...*

Sesión 1 · Del AVR al ESP32

→ ESP32 clásico → CPU de 32 bits (hasta 240 MHz), dos núcleos; Wi-Fi y Bluetooth; lógica GPIO de 3,3 V. Comparar con ATmega328P de 8 bits y Arduino Uno a 5 V.

→ DevKit V1 de las diapositivas: 30 pines. Algunas placas tienen otra distribución (p. ej. 38 pines) → mirar serigrafía y modelo exacto.

→ GPIO = entrada/salida de propósito general. Pin puede tener varias funciones: UART, PWM, SPI, etc. La GPIO matrix + IO_MUX ayudan a enrutar señales.

→ Capas vistas en la hoja: sketch Arduino → core Arduino-ESP32 → ESP-IDF → registros → hardware. Más arriba: simple; más abajo: control fino.

AVR	ESP32 (GPIO 0–31)
DDRx	GPIO_ENABLE_REG
PORTx salida	GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG
PINx	GPIO_IN_REG
pull-up	configuración IO_MUX

W1TS / W1TC: W1TS = Write 1 To Set → 1 escrito = encender ese bit. W1TC = Write 1 To Clear → 1 escrito = apagar. Los ceros escritos no afectan otros bits.

```
REG_WRITE(GPIO_OUT_W1TS_REG, 1UL << 16); // poner GPIO16 en 1
REG_WRITE(GPIO_OUT_W1TC_REG, 1UL << 16); // poner GPIO16 en 0
```

→ Ventaja: una escritura al periférico, sin leer–modificar–escribir el registro de salida. Una ISR u otro núcleo puede actuar entre esos pasos si no se hace de forma segura.

Mapa de pines · cuidado con los grupos

Grupo	Apunte rápido
GPIO16–19, 21–23, 25–27, 32–33	opciones útiles para LEDs/botones en ejemplos de clase
GPIO34–39	solo entrada; SIN pull-up / pull-down internas
GPIO0, 2, 5, 12, 15	strapping: se leen durante arranque; usar con cuidado
GPIO6–11	empleados por Flash SPI en el módulo WROOM de clase
GPIO1 TX0 / GPIO3 RX0	UART0, programación/Monitor Serie

→ Pulsador INPUT_PULLUP → usar GPIO21 en las prácticas; si se conecta en GPIO35, no funciona la pull-up interna, pin puede quedar flotante.

→ Alimentación: GND (tierra), 3V3 (salida regulada), VIN (según placa, entrada de alimentación). No confundir con GPIO.

→ ESP32 no es tolerante a 5 V en GPIO. LED con resistencia: R=(3,3 V−Vf)/I. Rojo Vf≈2 V y R=330 Ω → ≈4 mA.

→ UART: TX transmite, RX recibe. Serial.begin(115200) → configurar puerto serie para los mensajes...

↳ *Los motores necesitan más corriente que la de un GPIO → transistor/driver y fuente apropiada, no conectar directo.*

Arranque y estructura del sketch

→ Bootloader de ROM → primero; bootloader en Flash → después carga aplicación. ESP-IDF usa app_main(), FreeRTOS organiza tareas; Arduino corre setup() una vez y loop() repetidamente.

```
void setup() {
 pinMode(21, INPUT_PULLUP);
 Serial.begin(115200);
}
void loop() {
 // revisar entradas → lógica → salidas...
}
```

→ FreeRTOS = sistema operativo de tiempo real usado por ESP32. El sketch Arduino se ejecuta como tarea (loopTask).

Sesión 2 · Variables, desbordamiento y tipos

Tipo	Bits	Rango útil
uint8_t	8	0 … 255
int8_t	8	−128 … 127
uint16_t	16	0 … 65535
uint32_t	32	0 … 4 294 967 295
int32_t	32	−2 147 483 648 … 2 147 483 647

→ “u” = unsigned / sin signo. int mide 2 bytes en AVR Uno y 4 bytes en ESP32 clásico. Para tiempos y máscaras → tamaños fijos.

→ Desbordamiento uint8_t: 253 → 254 → 255 → 0 → 1. Aritmética sin signo da la vuelta módulo 256.

```
uint8_t a = 200, b = 100;
uint8_t c = a + b;  // 300 % 256 = 44
```

→ Promoción a int: uint8_t a=0xCC; ~a se aplica tras promover a entero. En ESP32 aparecen 32 bits al imprimir ~a en BIN; volver a 8 bits con (uint8_t)~a.

→ Máscara bit alto: 1UL << 31 (UL sin signo) en lugar de depender de un desplazamiento de int con signo.

→ const → no modificar; constexpr → calculable en compilación; static local → conserva valor entre llamadas; volatile → accesos que pueden cambiar por ISR/hardware. ¡volatile no significa atómico!

No confundir: uint8_t: 8 bits exactos; int: tamaño depende de plataforma. static conserva valor, pero no sobrevive al apagado. EEPROM/Flash sí (con las escrituras adecuadas).

Tiempo sin bloquear: millis() vs delay()

→ delay(500) detiene la tarea/loop durante la espera → si solo consulto botón en loop, puedo perder toques rápidos.

→ millis() devuelve milisegundos desde el arranque; leo el reloj y comparo sin detener la secuencia principal.

```
uint32_t ahora = millis();
if (ahora - tAnterior >= PERIODO) {
 tAnterior = ahora;
 // cambiar LED, revisar acción...
}
```

→ La resta con uint32_t funciona aun cuando millis() vuelve a cero, para intervalos de duración apropiada. millis() da la vuelta aprox. cada 49,7 días.

→ No guardar millis() en int firmado de 32 bits: después de ~24,8 días los valores dejan de encajar en ese tipo.

↳ *Mini ejemplo de clase: reloj de 8 bits → ahora 4 − antes 250 = 10 (módulo 256).*

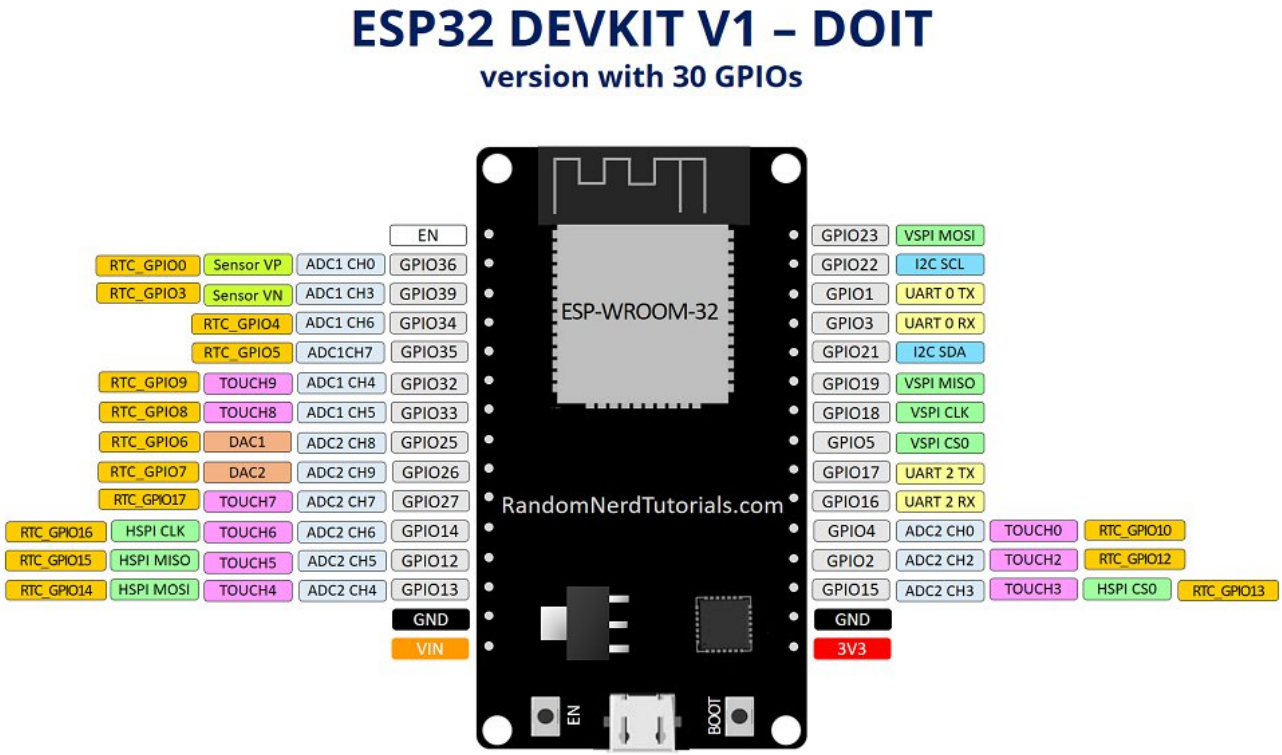
Botón, pull-up, nivel, flanco, rebote

→ INPUT_PULLUP → pulsador conectado entre GPIO y GND. Suelto = HIGH (1); presionado = LOW (0). Lógica invertida.

→ Nivel LOW dura muchas vueltas de loop si mantengo botón. Para contar un solo toque → detectar paso HIGH → LOW (flanco).

→ Rebote: el contacto mecánico oscila unos milisegundos. Filtrar estabilidad aprox. 30 ms en actividad de clase; evitar delay(200) como "solución".

```
bool lectura = digitalRead(21);
if (lectura != lecturaAnterior) {
  tUltimoCambio = millis(); lecturaAnterior = lectura;
}
if (millis() - tUltimoCambio >= 30 && lectura != estadoEstable) {
  estadoEstable = lectura;
  if (estadoEstable == LOW) pulsaciones++;
}
```



SEMANA 3 / TEMPORIZADORES DE HARDWARE

↳ *Clase → lo esencial, con abreviaturas y ejemplos. Algunas ideas quedan como en el cuaderno...*

Reloj, prescaler, tick, contador

→ Temporizador = periférico interno; cuenta pulsos por hardware. La CPU lo configura y el contador avanza sin que loop() sume cada tick.

→ En clase se usa APB = 80 MHz → 80 000 000 pulsos/segundo; cada pulso ≈12,5 ns.

→ Prescaler = divisor N. Ej.: N=80 → 80 000 000 / 80 = 1 000 000 ticks/s (1 MHz).

→ Tick = incremento del contador. Resolución = cuánto dura un tick. Alarma = valor objetivo que dispara aviso.

Fórmulas clave: f_tick = f_APB / N | resolución R = 1/f_tick | tiempo t = alarma × R | alarma = t × f_tick | N = f_APB/f_tick.

Dato	Cálculo rápido	Resultado
f=1 MHz, t=0,25 s	0,25 × 1 000 000	alarma 250 000
f=1 MHz, t=0,5 s	0,5 × 1 000 000	alarma 500 000
f=20 kHz, alarma=5000	5000 / 20 000	0,25 s
APB 80 MHz, f=20 kHz	80 000 000 / 20 000	N=4000
f=10 kHz, alarma=2500	2500 / 10 000	0,25 s

→ 1 Hz = 1 ciclo/s; 60 Hz = 60 ciclos/s → T = 1/60 s ≈16,7 ms. Relación de siempre: T=1/f.

→ Autorrecarga (auto-reload): contador alcanza alarma, se reinicia y vuelve a contar para repetir aviso. Desbordamiento: llega al máximo según cantidad de bits y da la vuelta; NO es lo mismo.

Timer API del core ESP32 3.x

→ API = funciones disponibles para manejar periféricos; core = paquete de la placa en Arduino IDE. Con versiones 3.x cambió la API: no copiar sin revisar ejemplos de core 2.x.

```
hw_timer_t *timer = nullptr;
volatile bool aviso = false;

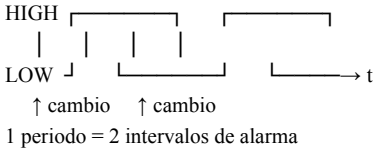
void IRAM_ATTR onTimer() {
  aviso = true;  // lo mínimo...
}

void setup() {
  timer = timerBegin(1000000);     // f_tick = 1 MHz
  timerAttachInterrupt(timer, onTimer);
  timerAlarm(timer, 250000, true, 0);  // 250 ms, repetir
}
```

→ Callback: función que escribo y entrego a la API para que se ejecute cuando ocurra el evento. ISR = rutina que atiende la interrupción.

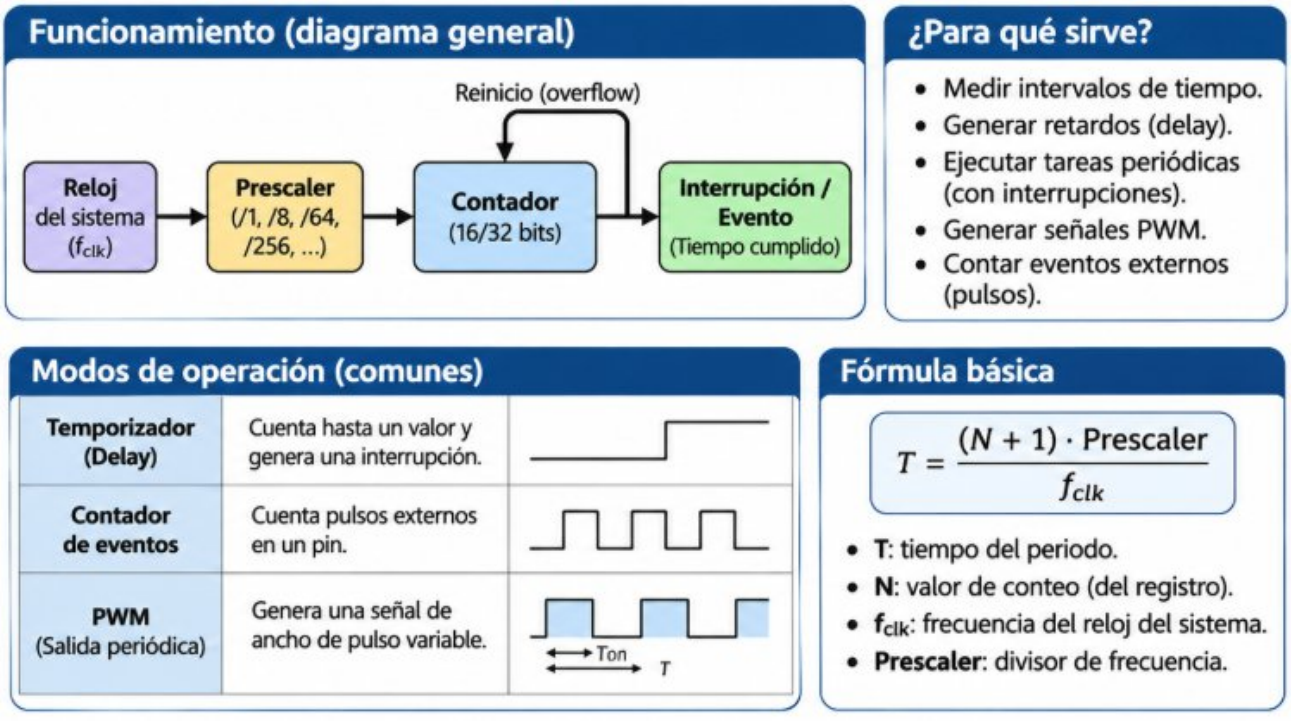
→ volatile bool aviso; → la ISR levanta bandera, loop() hace trabajo lento. No Serial.print dentro de la ISR.

Onda cuadrada: dos cambios hacen un periodo



> **Factor 2:** Si invierto GPIO en cada alarma → alarma = f_tick / (2 × f_señal). No confundir frecuencia de cambios con frecuencia de ciclos.

Señal pedida	f_tick	Alarma por inversión
100 Hz	1 MHz	5000 ticks
1000 Hz	1 MHz	500 ticks
5000 Hz	1 MHz	100 ticks



SEMANA 4 / INTERRUPCIONES EXTERNAS, ISR Y REBOTE

↳ *Clase → lo esencial, con abreviaturas y ejemplos. Algunas ideas quedan como en el cuaderno...*

Sondeo vs interrupción

→ Sondeo (polling): loop() pregunta una y otra vez "¿cambió el pin?". Si hay delay largo, puede no leer una pulsación corta.

→ Interrupción externa: periférico GPIO detecta cambio del pin, solicita atención a CPU → pausa ejecución, corre ISR y luego regresa.

→ No implica "se guardan todos los eventos" de forma ilimitada. Bandera bool guarda que ocurrió algo, pero no cuántas veces.

Flancos con INPUT_PULLUP

Modo	Cambio	Con botón a GND
FALLING	HIGH → LOW	presionar
RISING	LOW → HIGH	soltar
CHANGE	ambos	presionar + soltar

→ Sin rebote, CHANGE puede detectar dos flancos por toque completo. Con rebote puede dispararse más veces.

→ En ESP32 GPIO34–39 sí pueden tener interrupciones de entrada, pero carecen de pull-up interna. Pines GPIO6–11 del módulo en Flash: no usar para botón.

attachInterrupt() y función ISR

```
volatile bool alerta = false;
void IRAM_ATTR isrBoton() {
 alerta = true;
}

void setup() {
 pinMode(21, INPUT_PULLUP);
 attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);
}
```

→ attachInterrupt(pin, callback, FALLING) → registrar la función. Escribir isrBoton (sin paréntesis), porque no se ejecuta manualmente al configurarla.

→ ISR breve: cambiar bandera o sumar contador, terminar. Evitar delay, Serial.println, procesamiento pesado. Si tarda demasiado → latencia, watchdog, posible reinicio.

→ Watchdog = mecanismo que detecta tareas/ISR atascadas o sin progreso cuando deberían responder.

Flash, caché, IRAM, DRAM, volatile

→ Flash → programa guardado aunque apague. Caché → acelera lecturas. IRAM → RAM interna para instrucciones; DRAM → RAM interna para datos.

→ IRAM_ATTR → colocar código de ISR en memoria interna de instrucciones (según configuración). Útil cuando caché de Flash no está disponible.

→ Atención: si la ISR usa funciones o datos de lugares inaccesibles cuando se deshabilita caché, el atributo por sí solo no la vuelve completamente segura.

→ volatile indica que una variable puede cambiar por ISR/hardware y que el compilador debe respetar sus accesos. No garantiza atomicidad ni acceso seguro entre núcleos.

Variable compartida	¿volatile? / motivo
alerta (ISR escribe, loop lee)	sí
pulsos (ISR incrementa, loop lee)	sí; además sincronizar accesos compuestos
tUltimo (solo ISR)	no necesariamente
total (solo loop)	no por ISR

Bandera ≠ contador

```
volatile bool alerta = false;
void IRAM_ATTR isrBoton() { alerta = true; }
// ... 3 toques sin que loop revise → sigue siendo true (1 aviso)
```

→ Si necesito saber cuántos toques reales ocurrieron, ISR incrementa uint32_t pulsos; lectura/copia/reset compartidos → proteger correctamente.

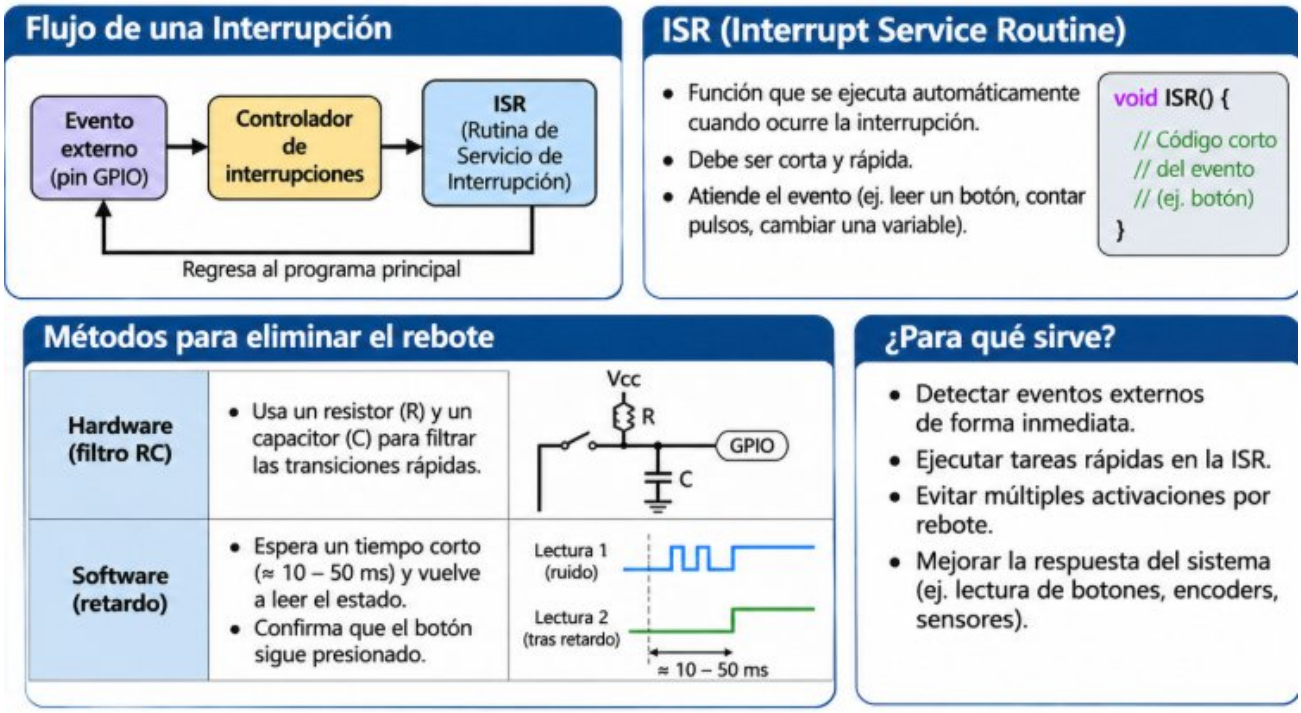
→ Patrón recomendado en clase: ISR levanta bandera y termina; loop baja bandera y hace Serial.print, LED, etc.

Flujo completo: Botón → periférico GPIO → interrupción → ISR breve (IRAM/volatile) → bandera o contador → loop procesa.

Rebote dentro de ISR

```
uint32_t tUltimo = 0; // solo la ISR
const uint32_t T_REBOTE = 50;
void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
```

→ Si un flanco aparece 10 ms después del último aceptado y T_REBOTE=50 → 10 < 50, se ignora. NO usa delay(40) para esperar.



APUNTES RAPIDOS

Codigo para recordar

```
REG |=  (1 << n);       // encender bit
REG &= ~(1 << n);       // apagar bit
REG ^=  (1 << n);       // alternar bit
if (REG & (1 << n))    // revisar bit

GPIO_OUT_W1TS_REG      // escribir 1 → set
GPIO_OUT_W1TC_REG      // escribir 1 → clear

ahora - anterior >= periodo   // millis sin bloquear

attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);
```

Preguntas

→ ¿Qué hace cada bus? → dirección, contenido, señal leer/escribir.
→ ¿Qué memoria permanece sin corriente? → Flash/EEPROM; SRAM no.
→ ¿Qué se lee al presionar con pull-up? → LOW (0).
→ ¿Qué hace 1UL << 31? → máscara sin signo de bit 31.
→ ¿Por qué uint8_t 255 + 1 es 0? → módulo 256.
→ ¿Por qué no usar GPIO35 con INPUT_PULLUP? → sin resistencia interna.
→ ¿Qué es autorrecarga? → repite al llegar a la alarma, no desbordamiento.
→ ¿Qué frecuencia sale si alterno cada 500 µs? → 1 kHz (dos alternancias/ciclo).
→ ¿Qué pasa con 3 toques y una bandera bool? → solo queda marcado 1 aviso.
→ ¿Por qué evitar Serial en ISR? → puede tardar/bloquear; loop lo hace después.